

Dang Lifestyle Operations Hub — Summary Specification

MadriBuild Demo Application

Route base: /inventory-app

Version: 1.0 (September 2026)

1. Product Overview

Name: Dang Lifestyle Operations (branded in-app as *Dang Lifestyle Operations Hub*)

Purpose: A single operations workspace for a US-based lifestyle brand, replacing spreadsheet/email workflows for product barcodes, inbound shipments (ASNs), damage claims, and refunds.

Positioning:

Operations hub for barcodes, ASN builder, damage claims, and refund tracking — demo/simulated Shopify and 3PL.

2. Goals & Problem Statement

Current Process (Before)	Proposed Process (In-App)
Spreadsheet product list	Single operations dashboard
Manual Shopify updates	Fix barcodes in one place
Email ASN to 3PL	Validate & generate ASN XLSX
Back-and-forth on damage claims	Track claims with 3PL workflow
Separate refund tracking	Refund status in claim detail

Primary outcome: Give ops teams one place to see what needs attention and move products, shipments, and claims forward without context-switching.

3. Users & Access Model

Role	Access Today
Ops user (demo)	Public read/write to operations collections (no login required)
Admin	Can reach app via /admin/apps; operations data is not admin-gated

Note: This is a **public demo** configuration. Production would require authentication and tighter Firestore rules.

4. Module Specification

4.1 Operations Dashboard (/inventory-app)

Purpose: Daily command center.

Metrics displayed:

- Total products
- Missing barcodes
- Pending/incomplete ASNs
- Open claims
- Refunds pending (approved claims with Shopify refund still pending)
- Orders requiring attention (new claims + ASNs with blocking validation errors)

Components:

- Stat cards (linked to filtered views)
- Action Required queue (prioritized links)
- Current vs. proposed process comparison

Data sources: `inventoryItems` , `asns` , `damageClaims`

4.2 Product & Barcode Manager (`/inventory-app/products`)

Purpose: SKU catalog with barcode readiness and Shopify sync state.

Product fields (`InventoryItem`):

- **Identity:** `id` , `name` , `sku`
- **Inventory:** `stock`
- **Physical:** `weightKg` , `dimensions` , `cartonWeightKg` , `cartonDimensions`
- **Media:** `photoUrl`
- **Barcode:** `barcode` , `barcodeStatus` (`ready` | `missing` | `verified`), `barcodeValue`
- **Shopify:** `shopifySyncStatus` (`synced` | `pending` | `error` | `not_linked`), optional `shopifyVariantId`
- **Timestamps:** `createdAt` , `updatedAt`

Filters: All · Missing barcode · Needs verification · Shopify sync pending

Product detail actions (`/inventory-app/products/items/:itemId`):

- Generate demo barcode (EAN-style)
- Mark barcode verified
- Simulate Shopify sync
- Print labels / product details
- Export · Edit · Delete SKU

Supporting routes:

- Create/edit SKU: `/products/items/new` , `/products/items/:itemId/edit`
 - Batch labels: `/products/labels`
-

4.3 ASN Builder (`/inventory-app/asn`)

Purpose: Build and validate Advance Shipment Notices for 3PL inbound.

ASN lifecycle statuses: `draft` → `validated` → `generated` → `submitted`

Header fields:

- Warehouse (US demo: Dallas TX, Reno NV, Allentown PA)

- Shipment date, PO reference
- Carrier (USPS, UPS, FedEx)
- Tracking reference, expected arrival

Line items: Pulled from products — SKU, barcode, name, quantity, carton weight/dimensions.

Validation rules (client-side):

- **Errors:** missing SKU, missing barcode, zero quantity
- **Warnings:** unverified barcode, duplicate SKU in same ASN, incomplete header fields

Workflow:

1. Compose ASN (new or edit existing)
2. Run validation → status `validated`
3. Generate XLSX export → status `generated`
4. Submit to 3PL (simulated delay) → status `submitted`

Export format: XLSX with columns for SKU, barcode, description, qty, carton weight, dimensions, PO ref, expected arrival.

4.4 Damage & Refund Center (`/inventory-app/claims`)

Purpose: Track customer damage/missing/wrong-item claims through resolution and refund.

Claim fields (`DamageClaim`):

- **IDs:** `id` , `displayId` (e.g. `DL-1026`)
- **Order/customer:** `orderNumber` , `customerName`
- **Product:** `productId` , `productName` , `quantity`
- **Issue:** `issueType` (`damaged` | `missing` | `wrong_item`)
- **Workflow:** `status` , `responsibility` , `resolution`
- **Evidence:** array of { `id`, `label`, `url` }
- **Refund block:** customer/shipping amounts, Shopify + claim refund status, customer notified flag

Status workflow:

`new` → `investigating` → `waiting_3pl` → `approved` → `refunded` → `closed`

Claim detail features:

- Status stepper with forward actions
- Set responsibility (customer, carrier, 3PL, warehouse, unknown)
- Set resolution (refund, replacement, store credit, reject)
- Request 3PL investigation (simulated) + copyable summary text
- Refund tracker (Shopify refund status, amounts, notification flag)

4.5 Operations Tasks (`/inventory-app/tasks`)

Purpose: Todoist-style task list for ops follow-ups.

Task fields (`OperationsTask`):

- `title` , `description` , `completed`
- `priority` (P1–P4, integers 1–4)
- `dueDate` (ISO date string, optional)

- `project` : `inbox` | `barcodes` | `asn` | `claims` | `refunds`
- Timestamps: `createdAt` , `updatedAt`

Views (query param `?view=`): `Inbox` · `Today` · `Upcoming` · `Completed`

Actions: `Quick-add` · `Toggle complete` · `Change priority/project` · `Delete`

5. Navigation & Shell

Layout: Collapsible sidebar + top bar (MadriBuild logo, theme toggle, live clock).

Module	Route
Dashboard	<code>/inventory-app</code>
Products	<code>/inventory-app/products</code>
ASN Builder	<code>/inventory-app/asn</code>
Claims	<code>/inventory-app/claims</code>
Tasks	<code>/inventory-app/tasks</code>

Quick action: New SKU → `/inventory-app/products/items/new`

Responsive behavior: Mobile drawer sidebar, collapsible desktop sidebar (persisted in `localStorage`).

6. Data Architecture

Firestore Collections

Collection	Document ID	Purpose
<code>inventoryItems</code>	matches <code>id</code> field	Product/SKU catalog
<code>asns</code>	matches <code>id</code> field	Advance shipment notices
<code>damageClaims</code>	matches <code>id</code> field	Damage/refund claims
<code>operationsTasks</code>	UUID (36 chars)	Ops task list

Security Rules (Demo)

- **Read:** Public for all four collections
- **Create/update:** Must pass schema validators; `id` must match doc ID; `createdAt` immutable on update
- **Delete:** Public (demo)

Client Architecture

Pages → Zustand stores → `*Data.ts` libs → Firestore

Stores: `useInventoryStore` · `useAsnStore` · `useClaimsStore` · `useTasksStore`

7. External Integrations

Integration	Status
Shopify	Simulated (~1.5s delay)
3PL	Simulated (ASN submit + claim investigation, ~1.2s)
Firebase App Check	Required when enforcement enabled
XLSX export	Real client-side generation via xlsx library

No live Shopify Admin API, 3PL API, or carrier tracking integrations in current scope.

8. Demo Data

Seed command: `npm run seed:operations`

Seeds (US demo):

- ~10 products (mix of missing/verified barcodes, varied Shopify sync states)
- ~3 ASNs (draft through submitted)
- ~10 damage claims (mixed statuses/resolutions)
- ~8 operations tasks

Demo constants: US warehouses, carriers (USPS/UPS/FedEx), US customer names, UPC-style barcodes.

9. Tech Stack

Layer	Choice
UI	React 19, TypeScript, Tailwind CSS
Routing	React Router 7
State	Zustand (atomic selectors)
Backend	Firebase Firestore
Build	Vite
Barcodes/labels	JSBarcode, QRCode, print CSS
Export	SheetJS (xlsx)

10. Non-Functional Requirements

- **Performance:** Client-side filtering/sorting; hydrate-on-demand per module
 - **Accessibility:** Sidebar tooltips when collapsed, keyboard escape closes mobile drawer
 - **Local dev:** App Check debug token + API key HTTP referrers for localhost
 - **Deployment:** `npm run deploy` (hosting + Firestore rules)
-

11. Out of Scope (Current Build)

- User authentication for operations modules
 - Real Shopify / 3PL / carrier API connections
 - Real-time Firestore listeners (uses fetch-on-hydrate)
 - Tasks on dashboard metrics/action queue
 - Legacy `InventoryCatalogPage.tsx` (superseded by `ProductBarcodePage.tsx`)
-

12. Success Criteria (Demo)

1. Ops user can see at-a-glance what needs attention from the dashboard.
 2. Missing barcodes are identifiable and fixable in Product Manager.
 3. ASNs can be validated and exported as XLSX before simulated 3PL submission.
 4. Claims move through a defined status workflow with refund tracking.
 5. Tasks capture follow-ups tied to ops projects (barcodes, ASN, claims, refunds).
-

Generated for MadriBuild — Dang Lifestyle Operations Hub